RAG Based Interactive Chatbot for Video Streaming Services

Hamza Gözükara
Mindsane Ltd. Co.
Researh & Development Department
İstanbul, Türkiye
gozukarah@mef.edu.tr

Jay Patel
SST-TEK
Research & Development Center
İstanbul, Türkiye
jayyypatel24@gmail.com

Erkan Kara
Demirören Media Inc.
Research & Development Center
İstanbul, Türkiye
erkan.kara@demirorenmedya.com

Ayşenur Yıldız
Demirören Media Inc.
Research & Development Center
İstanbul, Türkiye
aysenur.yildiz@demirorenmedya.com

Ozan Köseoğlu
D-Smart Inc.
Research & Development Center
İstanbul, Türkiye
Ozan.Koseoglu@demirorenmedya.com

Didem Makaroğlu
Insider Inc.
Research & Development Center
İstanbul, Türkiye
didem.makaroglu@useinsider.com

Yassine Drias
MEF University
Computer Engineering Department
İstanbul, Türkiye
driasy@mef.edu.tr

Şirin Özlem
MEF University
Industrial Engineering Department
İstanbul, Türkiye
ozlems@mef.edu.tr

Tuna Çakar
MEF University
Computer Engineering Department
İstanbul, Türkiye
cakart@mef.edu.tr

2025 10th International Conference on Computer Science and Engineering (UBMK) | 979-8-3315-9975-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/UBMK67458.2025.11206833

***Abstract*—The proliferation of content within video streaming services presents a significant challenge for users seeking person alized recommendations and specific information. This research addresses this challenge by developing a Retrieval-Augmented Generation (RAG) chatbot designed to enhance user experience through conversational AI. The primary contribution of this work is a novel Retrieval-Augmented Generation (RAG) archi tecture featuring a dual-retrieval system that combines semantic search for descriptive requests and structured queries for fact based inquiries. This approach grounds the Large Language Model (LLM) in a factual knowledge base, mitigating the risk of hallucinations. The system is engineered to handle empty data retrieval scenarios by dynamically relaxing search filters, ensuring a robust user experience. The effectiveness of this RAG approach was validated through a comprehensive set of automated evaluations. The system demonstrates high precision in ranked list retrieval with questions like "Recommend me the top 5 action movies with highest IMDb scores", achieving an average NDCG@k of 0.837. While the chatbot shows strong semantic understanding by achieving 91% accuracy with contextual clues such as "Which Batman movies are directed by Christopher Nolan?", its performance with more ambiguous, plot-only queries (59.5% accuracy) indicates clear opportunities for future refinement. T hese r esults c onfirm th at th e dual-tool architecture successfully combines the flexibility o f semantic search with the precision of structured queries, paving the way for more intuitive and efficient c ontent d iscovery o n streaming platforms.**

***Keywords*——large language models, retrieval augmented gen eration, interactive assistant, embeddings, movie recommenda tion.**

I. Introduction

The digital age has ushered in an era of unprecedented content availability, particularly within video streaming services, which now boast vast and continuously expanding libraries of movies and TV series. While this abundance offers diverse entertainment options, it simultaneously presents a significant challenge: content discovery and information overload. Users often find it difficult and time-consuming to navigate these extensive catalogs, leading to frustration and missed opportunities for engaging with relevant content. Traditional recommendation systems, though effective to a degree, fre-quently rely on static algorithms that can lead to filter bubbles or lack the nuanced interactivity required for truly personalized content exploration [1]. Traditional recommen-dation systems—including collaborative filtering, content-based filtering, and hybrid models—offer partial solutions but often reinforce users' existing preferences, limiting nov-elty and deeper exploration [2]. Addressing this, researchers have proposed counterfactual and interactive frameworks to burst filter bubbles and improve long-term satisfaction [3]. Conversational Artificial Intelligence (AI), powered by Large Language Models (LLMs), comes with a compelling alternative by enabling more natural, intuitive, and context-aware interaction. However, LLMs alone can suffer from factual inaccuracies or "hallucinations" particularly when responding to precise queries in specialized domains. They also lack the ability to access real-time information or a specific, proprietary knowledge base, such as a streaming service's current content library. This limitation means they cannot account for newly added titles and may recommend content that is not actually available on the platform, making them unreliable for this use case on their own. Therefore, a specialized architecture is required to ground the LLM in a dynamic, fact-based context.

Retrieval-Augmented Generation (RAG) is a framework designed to specifically mitigate these issues. RAG enhances

979-8-3315-9975-1/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: The University of Toronto. Downloaded on February 12,2026 at 18:39:40 UTC from IEEE Xplore. Restrictions apply.

the capabilities of LLMs by connecting them to an external, authoritative knowledge base which can be updated anytime. Before generating a response, the system first retrieves relevant data snippets related to the user's query. This retrieved information is then provided to the LLM as context, effectively grounding the model's response in factual data and significantly improving its accuracy and reliability [4].

In their work, Aditya, Kushagra and Dinesh [5] proposed a movie recommendation chatbot that utilizes a content-based filtering approach driven by cosine similarity and NLP tools. Unlike traditional systems that rely on historical user data and collaborative filtering, their model aims to solve the "cold start" problem by directly interacting with the user. The system prompts the user for their current preferences—such as genre, actors, and keywords—and compares this input against a "word soup" created from each movie's metadata. By vectorizing this text and calculating cosine similarity, the chatbot recommends the most relevant content based on the user's immediate input. While this approach successfully provides recommendations without prior user history, the authors note a key limitation: the computational cost of creating the pairwise similarity matrix required them to significantly reduce their dataset size, highlighting a trade-off between personalization and scalability.

Bhat et al. [6] collaborated with Jerry Gao to develop a RAG-enhanced restaurant chatbot tailored for post-COVID customer interaction challenges. They constructed a Neo4j knowledge graph from restaurant data and applied TF-IDF embeddings to traverse the graph and select relevant tokens in response to user queries. These tokens, combined with the user's input, were fed into a T5 model to generate context-aware, nuanced answers. Bhat et al. reported a BLEU score of 0.60, demonstrating strong linguistic precision, and performed AI-testability evaluations at word, sentence, and information levels. Their findings showed that the RAG-based system produced more knowledgeable, diverse, and relevant responses than state-of-the-art baselines, with information scores ranging between 0.6 and 0.8.

In their study, Reddy, Patra, and Mishra [7] proposed and implemented a modular AI-based chatbot framework that integrates Retrieval-Augmented Generation (RAG) with a recommender system to enhance context-aware conversational support. The primary development and implementation responsibilities were undertaken by Reddy, who designed the system architecture and developed the core functionalities using open-source tools such as LangChain, FAISS, Ollama, and classical NLP libraries like NLTK and SpaCy. The framework supports ingestion of diverse document types, performs semantic search over vector embeddings, and employs local LLMs for response generation. Patra and Mishra contributed to the theoretical formulation of the framework, guided the selection and integration of system components, and oversaw the experimental validation of the chatbot's effectiveness in terms of response accuracy, user satisfaction, and recommendation relevance.

The main purpose of this research is to propose and develop a Retrieval-Augmented Generation (RAG) chatbot designed to overcome these challenges in the context of movie and series recommendations and information retrieval. The RAG architecture is particularly well-suited for this domain as it combines the powerful generative capabilities of LLMs with a robust information retrieval mechanism, ensuring that responses are grounded in factual data from a proprietary content database. This approach mitigates the risk of hallucinations by actively retrieving relevant information before generating a response [8], thereby improving accuracy and user trust.

II. Methods

A. Dataset Description

The dataset provided to us by D-Smart Incorpration, a Pay-Tv company in Türkiye. The provided data is used to create our retrieval system's knowledge base, it contained 2822 movies and 703 series content, 3525 contents in total. As illustrated in Fig. 1, it consists of several metadata about the series and movies. To enrich the existing content in our database, additional metadata was collected from IMDb using Python's 'Selenium' library. This scraping process retrieved key attributes such as IMDb ratings, the number of user reviews (as an indicator for popularity), film summaries, and associated keywords.

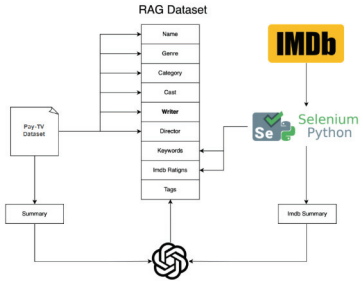

Fig. 1: RAG Dataset Structure

Notably, the dataset included two distinct summary fields after scraping. To enhance the keyword-based retrieval performance of our system, we employed OpenAI's gpt-4.1-mini to extract salient keywords and tags from both summaries. This specific model was deliberately chosen as it provided the optimal balance of performance, cost, and efficiency for this task. While keyword extraction requires robust language understanding, it does not necessitate the advanced reasoning capabilities of larger, more expensive models, making gpt-4.1-mini a sufficient and cost-effective choice for processing the entire dataset. The extracted terms were then integrated into the existing keyword field of our database.

B. Creating Embeddings

Throughout this study, our selection of embedding models evolved iteratively to best suit the growing complexity and richness of our content metadata. This iterative approach allowed us to balance performance with computational cost at each stage of development. Initially, for the proof-of-concept phase with a simpler dataset, we employed the *sentence-transformers/all-MiniLM-L6-v2 model* with 372 dimensions.

Authorized licensed use limited to: The University of Toronto. Downloaded on February 12,2026 at 18:39:40 UTC from IEEE Xplore. Restrictions apply.

This lightweight and efficient model was ideal for rapid prototyping, where the primary need was to quickly embed basic metadata. However, as our metadata pool expanded with more detailed and nuanced information, it became apparent that upgrading to more dimensional model were crucial. To address this, we transitioned to *OpenAI/text-embeddings-3-mini* with 1536 dimensions. With its larger vector dimensionality, this model offered a greater capacity to capture the finer semantic distinctions in our more detailed, structured metadata without being prohibitively expensive. This upgrade was a necessary step to improve the granularity of our semantic search capabilities. The final and most significant upgrade was to the *OpenAI/text-embeddings-3-large model* with 3072 dimensions. This decision was made directly after integrating extensive, unstructured data scraped from IMDb, such as lengthy plot summaries and a wide array of user-generated keywords. Representing the complex narratives and diverse terminology within this long-form text required the high dimensionality that *text-embeddings-3-large* provides. Its larger vector space is crucial for preventing information loss and accurately capturing the semantic depth of the expanded content, which is essential for handling complex, descriptive user queries.

Via gathering all of the values of the fields illustrated in Fig. 1 in a single string for each content then embedding it, vector representations for all of the contents are created. To efficiently store and search these high-dimensional vectors, Pinecone, a vector database optimized for fast and scalable similarity search, is used in this study, as in many other RAG studies [9, 10, 11]. Pinecone allowed us to index and retrieve semantically similar content in real-time, enabling low-latency recommendations and queries. Its support for metadata filtering and hybrid search also made it easier to integrate additional constraints such as filtering by content type or tags, which was critical for delivering more personalized and relevant results in our retrieval system. Pinecone offers three different similarity metrics for vector comparison: dot product, Euclidean distance, and cosine similarity. For this study, we selected cosine similarity, as its use is a widely adopted practice for semantic search tasks involving modern text embeddings [12]. This metric is particularly effective because it measures the angle between two vectors, focusing on their orientation (i.e., semantic meaning) rather than their magnitude. This method aligns with the design of transformer-based embedding models, which represent a text's meaning in the vector's direction. This allows the system to accurately identify conceptually similar content, ensuring that retrieval is optimized for semantic relevance.

C. Retrieval Mechanisms

The core of our RAG system's ability to provide both flexible recommendations and precise information lies in its dual retrieval mechanism, leveraging two distinct databases tailored to different query types: a Pinecone vector database for semantic similarity searches and a MongoDB collection for structured, exact-match queries.

This hybrid approach ensures comprehensive and accurate content retrieval based on user intent. Fig. 2 illustrates the workflow behind the assistant's response to the user, including the usage of the retrieval tools. As it suggests (and will be mentioned later), the assistant can call its tools until it decides no retrieval is needed before its answer.

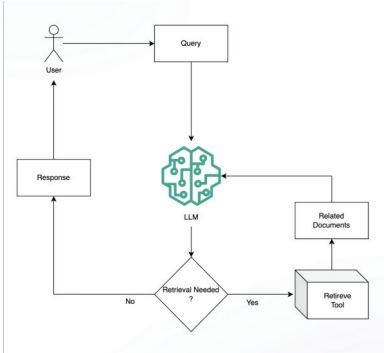

Fig. 2: Chatbot Response Workflow

For open-ended queries, descriptive requests, and broad recommendations, our system utilizes the Pinecone vector database. After the content metadata is embedded into high-dimensional vectors as described in the "Creating Embeddings" section, these vectors are stored and indexed in Pinecone. When a user submits a query (e.g., "Recommend an interesting sci-fi movie" or "Films about a group of thieves stealing ideas through dreams"), the user's input is also embedded. A similarity search is then performed in Pinecone to identify content vectors most semantically similar to the query vector. This process allows the system to retrieve movies and series that conceptually match the user's request, even if exact keywords are not present in the original query.

This semantic retrieval is primarily orchestrated by the PineconeSearchTool within our LangChain agent. This tool is designed to construct a well-structured query to the Pinecone database, incorporating keywords extracted from the user's input and applying various filters (e.g., *Type, Genre, Cast, Directors, Writers, Imdb_Rating, NumberOfReviews*) when specified by the user. A crucial feature of this tool is its robust error handling and retry mechanism: if an initial pinecone_search call returns no results, the agent is instructed to automatically attempt a new query by strategically relaxing the most restrictive filters (prioritizing *NumberOfReviews* and *Imdb_Rating*). This ensures that the system provides results even for highly constrained initial requests, broadening the search to find relevant content when exact matches are unavailable. The retrieved documents, containing detailed metadata, are then passed to the LLM for response generation.

To complement the semantic search capabilities, the system also incorporates a MongoDB collection for queries requiring precise filtering, sorting, or exact matches. This database stores the raw, structured metadata of all content items. The *MongoDBQueryTool* is specifically designed for scenarios where users explicitly request ranked lists (e.g., "top 10 movies by IMDb rating," "most popular series of Christopher Nolan") or specific content details based on exact attribute values (e.g., "Which actors played in 'The Dark Knight' directed by Christopher Nolan?").

Authorized licensed use limited to: The University of Toronto. Downloaded on February 12,2026 at 18:39:40 UTC from IEEE Xplore. Restrictions apply.

This tool constructs MongoDB queries dynamically based on user intent, allowing for operations such as sorting by *Imdb_Rating* or *NumberOfReviews* in ascending or descending order, and limiting the number of results. Filters for Name, Type, Genre, Cast, Directors, and Writers are applied directly to the MongoDB collection. The output from the *MongoDBQueryTool* provides highly accurate and specific content information, which is then fed into the LLM to formulate a precise and informative response. The combination of these two distinct retrieval strategies allows our RAG chatbot to proficiently handle a wide spectrum of user inquiries, from vague recommendations to highly specific data requests.

D. Conversation History

Effective conversational AI requires the ability to maintain context and continuity throughout an interaction. Our RAG chatbot addresses this through the implementation of conversation history management, crucial for delivering personalized and coherent responses over multi-turn dialogues [13]. The system leverages LangChain's *ConversationSummaryMemory*, which is specifically configured to build and refine a concise summary of the ongoing conversation. The ConversationSummaryMemory employs a dedicated, faster Large Language Model (*gpt-4.1-nano* in this case) and a custom prompt to continuously update the conversation summary. A critical instruction within this summarizer LLM's prompt ensures that previous suggestions of the assistant and any questions posed by the assistant to the user are explicitly included in the summary, thereby guiding future responses and maintaining conversational flow.

By synthesizing past turns into a compact yet informative summary, the chatbot can understand the evolving context of the user's preferences and previous recommendations without needing to process the entire raw chat log each time. This summarized history is then injected into subsequent prompts sent to the main LLM (*gpt-4.1-mini*), enabling the agent to make informed decisions about tool usage and response generation, ensuring that recommendations and information provided are consistent with prior interactions and stated preferences.

E. Prompt Engineering

Effective prompt engineering is paramount in guiding the Large Language Model (LLM) to perform its intended functions within the Retrieval-Augmented Generation (RAG) framework [14, 15], ensuring accurate tool utilization, coherent response generation, and strict adherence to the chatbot's defined persona. In this study, prompt engineering primarily involves crafting a structured prompt template that dictates the interaction between the user, the LLM, and the retrieval tools. The prompt template is constructed using a series of distinct message types that provide explicit instructions and contextual information to the LLM:

The "Initial System Instructions" serve as the chatbot's core identity, defining it as a movie and TV show recommendation assistant. These instructions guide the Large Language Model (LLM) on how to intelligently use retrieval tools, distinguishing between semantic search and structured database queries. They also dictate how the LLM should interpret and present tool outputs. A crucial adaptive feature is included, where the agent automatically relaxes restrictive filters like popularity or rating thresholds if an initial semantic search yields no results.

The core of the prompt is built around several placeholders that provide the necessary context for the LLM's reasoning process for each interaction. Conversation History dynamically integrates the summarized conversation from *ConversationSummaryMemory*. Another placeholder for the Current User Input, which inserts the user's immediate query for the LLM to process. Finally, a crucial placeholder for the agent's internal "Scratchpad" contains its reasoning steps, including the LLM's thought process, the calls made to external tools, and the subsequent outputs from those tools.

Prompt injection is a form of adversarial attack in which maliciously crafted inputs are used to manipulate the output behavior of Large Language Models (LLMs), often causing them to deviate from their intended functions. These attacks typically exploit the compositional nature of prompts in LLM-integrated applications by injecting payloads that override or nullify predefined instructions. [16] In order to avoid prompt injection for our chatbot, a concluding system instruction is applied; it reinforces the chatbot's core purpose as a movie and TV show recommendation system. This serves as a vital guardrail, ensuring strict adherence to the chatbot's domain and preventing any attempts to divert its function or persona. It explicitly states that the chatbot must always respond in the designated language (Turkish in this case) and consistently utilize its available tools when appropriate for content retrieval.

III. Results

This section presents the empirical outcomes of the developed RAG chatbot. It contains a representative user interaction, which illustrates how the retrieval tools handles complex queries and a detailed quantitative evaluation of the chatbot's performance on various retrieval and ranking tasks.

In the user interaction shown in Fig. 3, the user asks for two movie recommendations directed by Christopher Nolan and starring Christian Bale, with an IMDb rating between 7 and 9, at least 10,000 reviews, and including themes of detective work, heroism, and mystery solving. This user-query is a pre-prepared question in order to get directly these two contents, "The Dark Knight" and "Batman Begins", and as expected, the assistant recommends these two contents via first extracting necessary filters from the user's query, then calling its semantic search tool using these filters with the query containing the requested themes.

The arguments of the semantic search tool for the execution in Fig. 3 can be seen in Fig. 4. This scenario demonstrates the chatbot's ability to interpret multiple user constraints and use the semantic search tool effectively. The LLM successfully extracted relevant filters and presented the

Authorized licensed use limited to: The University of Toronto. Downloaded on February 12,2026 at 18:39:40 UTC from IEEE Xplore. Restrictions apply.

top recommendation from the tool's output in a conversational and informative manner.

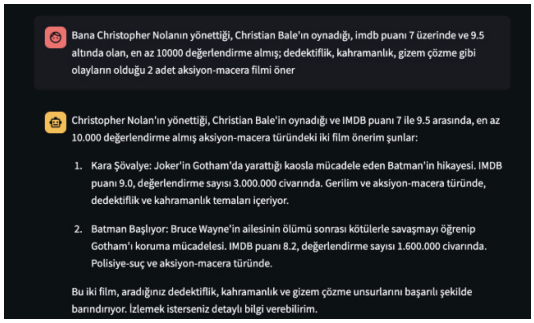

Fig. 3: Example User Interaction

Fig. 4: Tool Call Arguments

To systematically assess the chatbot's retrieval accuracy and ranking quality, a series of automated evaluations were conducted. On average, the chatbot took 11.7 seconds to generate a response during these tests. These evaluations targeted the system's ability to handle specific information retrieval, semantic understanding of storylines, and ranked list generation. The key outcomes of the automated tests are detailed in Table. I.

TABLE I: Performance Evaluation Summary of the RAG Chatbot

Category	Metric	Score	# Tests
Semantic Search	Accuracy (w/ clues)	0.910	200
	Avg. Rank (w/ clues)	1.23	200
	Accuracy (plot-only)	0.595	200
	Avg. Rank (plot-only)	1.65	200
Ranked Lists	Exact Order Match	0.730	40
	Precision@k	0.801	40
	NDCG@k	0.837	40
Exact Match	Accuracy	0.885	200

Semantic Content Retrieval with and without Explicit Clues phase contains two distinct scenarios that were tested to evaluate the semantic retrieval capabilities of the chatbot. An agent generated questions that provided contextual clues about a specific content without mentioning its name (e.g., "What was the name of the film directed by Christopher Nolan about dream theft?"). The evaluation measured both exact match accuracy and the position of the correct content in the recommendation list. The chatbot demonstrated a high average accuracy of 0.91 out of 200 different tests in retrieving the correct content. On average, the correct content was ranked at position 1.23 within the top 5 recommendations.

A more challenging evaluation involved generating questions based solely on the plot of a content item, with no other identifying information (e.g., "Is there a film about a man who, after a stroke, writes his life story by blinking his one functioning eye?"). For these queries, the chatbot achieved an average accuracy of 0.595 out of 200 different tests. The average rank for the correct content was 1.65 out of 5 recommendations.

Ranked List Generation phase evaluates the accuracy of the chatbot's structured retrieval tool using a ground truth table of 40 questions requesting ranked content (e.g., "Can you recommend the top 5 action movies with the highest IMDb scores?"). The retrieved content IDs were compared against the pre-defined correct lists.

- **Exact Order Match Ratio:** The chatbot correctly reproduced the exact order of the top-ranked content in %73 of the queries.
- **Precision@k:** The average *Precision@k* was 0.801, indicating a high relevance of the retrieved items. The "k" value equals the numbers of retrieved contents (its 5 most of the cases) for the query.
- **NDCG@k:** The *NDCG@k* (Normalized Discounted Cumulative Gain) was 0.837, showing that the chatbot not only retrieves relevant items but also ranks them appropriately. The "k" value equals the numbers of retrieved contents.

Exact Information Retrieval phase is an evaluation was designed to test the chatbot's ability to retrieve a specific content entity when its name is explicitly mentioned in the query (e.g., "What is the cast of 'The Dark Knight'?"). This test focused on exact match accuracy. The chatbot achieved an average accuracy of 0.885 out of 200 different automated tests.

User Perception of Semantic Recommendations phase is for more general queries that do not refer to a specific content item (e.g., "Can you recommend some exciting movies?"), a user survey of 50 different question-retrieved content(s) pairs was conducted to evaluate the quality of the recommendations. 11 real users rated the recommendations on average 3.51, on a scale of 0 to 5. This qualitative feedback provided insights into user satisfaction with the chatbot's ability to handle broad semantic requests, complementing the automated evaluation metrics.

These comprehensive evaluations demonstrate the robustness of the dual-retrieval mechanism, highlighting both the precision of structured queries and the effectiveness of the semantic search in understanding user intent from various types of questions.

IV. Discussion And Conclusion

The empirical results and quantitative evaluations demonstrate the efficacy of our Retrieval-Augmented Generation (RAG) chatbot. The integration of a Pinecone vector database for semantic search and a MongoDB collection for structured queries proved to be a robust architectural choice. This dual-retrieval mechanism allows the chatbot to adeptly handle a wide spectrum of user queries, from broad, descriptive recommendations to precise, data-driven requests. Moreover, as can be seen in Fig. 3, the fact that retrieval tools can handle complex filters makes the assistant powerful in specific

Authorized licensed use limited to: The University of Toronto. Downloaded on February 12,2026 at 18:39:40 UTC from IEEE Xplore. Restrictions apply.

queries.

Our quantitative analysis provides insight into the specific strengths and weaknesses of the dual-retrieval architecture. The NDCG@k score achieved by the structured retrieval tool demonstrates more than just relevance; it shows the system excels at ranking content in a logical order that matches user expectations for ”top-k” queries. This high-fidelity ranking is a direct result of leveraging MongoDB’s precise sorting capabilities and is crucial for building user trust. In parallel, the semantic search tool’s strength on queries with contextual clues confirms its ability to effectively translate natural language descriptions into relevant content recommendations. However, the significant performance drop on abstract, plot-only queries reveals a key cause-and-effect relationship: The system’s semantic understanding, while powerful, performs optimally when queries contain specific named entities or keywords. The performance difference with more ambiguous queries highlights a clear avenue for future improvement, particularly in enhancing the model’s ability to interpret vague user intent.

These findings validate that our hybrid RAG approach effectively mitigates the risk of LLM hallucinations by grounding responses in factual data, thereby outperforming standalone language models in this domain. Furthermore, the strategic use of conversational summary memory was vital for maintaining context in multi-turn dialogues , and sophisticated prompt engineering, including the adaptive filter relaxation mechanism, significantly enhanced the system’s resilience and the overall user experience.

Despite the promising results, the system has certain limitations. The chatbot’s performance is inherently dependent on the quality and completeness of the metadata in its knowledge base. Additionally, as the complexity of retrieval tool filters increases, there is a growing risk of introducing ambiguity or confusion in the language model’s reasoning. When filters become too intricate or layered, the LLM may misinterpret the intent or fail to align the retrieved content correctly with the query. This introduces a trade-off: while advanced filtering can improve precision up to a point, pushing it too far may reduce overall system reliability. Lastly, a notable weakness is the system’s average response time of 11.7 seconds. This latency, while acceptable for some use cases, could impact user engagement in a real-time conversational setting. Future work should prioritize optimizing this aspect, perhaps by employing smaller models for certain tasks or implementing more efficient caching strategies.

While the current iteration demonstrates significant promise, future work will focus on continuous improvement. A critical next step is the active collection and analysis of user feedback to identify areas for refinement. Key areas for future development includes enriching the content metadata by scraping additional attributes, such as release dates, integrating new filtering options such as distinguishing between domestic and international content, to provide more granular control to the user, further optimizing our embedding and search strategies to enhance retrieval accuracy, particularly for the more challenging, ambiguous queries identified in our evaluations.

Acknowledgement

This study has been conducted under the Research and Development Center of D-Smart Inc supported by Ministry of Industry and Technology.

References

[1] W. Cai, F. Feng, Q. Wang, T. Yang, Z. Liu, and C. Xu, “A Causal View for Item-level Effect of Recommendation on User Preference,” WSDM’23, Feb. 2023, doi: https://doi.org/10.1145/3539597.3570461

[2] Y. Kim, S. Oh, C. Noh, E. Hong, and S. Park, “Design of a Serendipity-Incorporated Recommender System,” Electronics, vol. 14, no. 4, p. 821, Feb. 2025, doi: https://doi.org/10.3390/electronics14040821. Available: https://www.mdpi.com/2079-9292/14/4/821. [Accessed: May 07, 2025]

[3] C. Gao et al., “CIRS: Bursting Filter Bubbles by Counterfactual Interactive Recommender System,” arXiv.org, 2022. Available: https://arxiv.org/abs/2204.01266. [Accessed: Jun. 20, 2025]

[4] K. W. Church, J. Sun, R. Yue, P. Vickers, W. Saba, and R. Chandrasekar, “Emerging trends: a gentle introduction to RAG,” Natural Language Engineering, vol. 30, no. 4, pp. 870–881, Jul. 2024, doi: https://doi.org/10.1017/s1351324924000044. Available: https://www.cambridge.org/core/journals/natural-language-engineering/article/emerging-trends-a-gentle-introduction-to-rag/4FF461F4066A0C16135F2D2849E3356A

[5] A. Dixit, K. Gupta, and D. G, “Movie Recommendation Chatbot Using Cosine Similarity and NLP tools,” 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), pp. 1–7, Jul. 2023, doi: https://doi.org/10.1109/icccnt56998.2023.10307673.

[6] V. Bhat, Sree Divya Cheerla, J. R. Mathew, N. Pathak, G. Liu, and J. Gao, “Retrieval Augmented Generation (RAG) Based Restaurant Chatbot with AI Testability,” pp. 1–10, Jul. 2024, doi: https://doi.org/10.1109/bigdataservice62917.2024.00008

[7] T. Kumar, “Design and Implementation of an AI-Based Chatbot Framework with Retrieval-Augmented Generation and Integrated Recommender System for Interactive User Support,” Jan. 2025, doi: https://doi.org/10.2139/ssrn.5250507.

[8] P. Béchard and O. M. Ayala, “Reducing hallucination in structured outputs via Retrieval-Augmented Generation,” arXiv.org, Apr. 11, 2024. doi: https://doi.org/10.48550/arXiv.2404.08189.

[9] Y. Ke et al., “Development and Testing of Retrieval Augmented Generation in Large Language Models – A Case Study Report,” arXiv (Cornell University), Jan. 2024, doi: https://doi.org/10.48550/arxiv.2402.01733

[10] J. Owoyemi et al., “Open-Source Retrieval Augmented Generation Framework for Retrieving Accurate Medication Insights from Formularies for African Healthcare Workers,” medRxiv (Cold Spring Harbor Laboratory), Feb. 2025, doi: https://doi.org/10.1101/2025.02.20.25322640

[11] S. Ruangtanusak, N. Rungseesiripak, P. Rojratchadakorn, M. Charattrakool, and N. Nitarach, “DoTA-RAG: Dynamic of Thought Aggregation RAG,” arXiv.org, 2025. Available:

[12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Aug. 2019. Available: https://arxiv.org/pdf/1908.10084 https://arxiv.org/abs/2506.12571. [Accessed: Jun. 26, 2025]

[13] Q. Wang et al., “Recursively Summarizing Enables Long-Term Dialogue Memory in Large Language Models,” arXiv.org, Feb. 18, 2024. doi: https://doi.org/10.48550/arXiv.2308.15022. Available: https://arxiv.org/abs/2308.15022

[14] I. Papadimitriou, I. Gialampoukidis, S. Vrochidis, Ioannis, and Kompatsiaris, “RAG Playground: A Framework for Systematic Evaluation of Retrieval Strategies and Prompt Engineering in RAG Systems,” arXiv.org, 2024. Available: https://arxiv.org/abs/2412.12322?. [Accessed: Jun. 26, 2025]

[15] Z. He, H. Jiang, Z. Wang, Y. Yang, L. Qiu, and L. Qiu, “Position Engineering: Boosting Large Language Models through Positional Information Manipulation,” arXiv.org, 2024. Available: https://arxiv.org/abs/2404.11216

[16] Y. Liu et al., “Prompt Injection attack against LLM-integrated Applications,” arXiv.org, Jun. 08, 2023. doi: https://doi.org/10.48550/arXiv.2306.05499. Available: https://arxiv.org/abs/2306.05499

Authorized licensed use limited to: The University of Toronto. Downloaded on February 12,2026 at 18:39:40 UTC from IEEE Xplore. Restrictions apply.